An AI Based High-Precision Industrial Thermometer

Puni Aditya, Vivek K. Kumar, Jnanesh Sathisha Karmar, Burra Shreyas Goud, Dr. G.V.V. Sharma
Department of Electrical Engineering,
Indian Institute of Technology Hyderabad,
Kandi, India 502284
gadepall@ee.iith.ac.in

***Abstract*—In this paper, we design and develop an industrial thermometer using the PT100 temperature sensor. This is done by analyzing the suitability of five machine learning (ML) techniques: a quadratic/inverse quadratic Least Squares (LSQ) model, a Stochastic/inverse Stochastic Gradient Descent (SGD) trained model, and a non-linear Random Forest Regressor by calibrating them with respect to a commercial pen thermometer. The calibration process is also automated using Optical Character Recognition (OCR). Through numerical results, we conclude that the LSQ model is superior to the other models and implement it using the Arduino framework on an Arduino UNO.**

I. Introduction

Temperature measurement is a fundamental requirement in process control, automotive systems, and instrumentation. Among the various contact temperature sensors available, the Platinum Resistance Temperature Detector (RTD), commonly known as the PT100, is considered the standard for precision thermometry [1].

II. PT-100 Fundamentals

The PT100 is defined by a nominal resistance of $100\,\Omega$ at 0°C. Unlike thermocouples which generate a voltage, RTDs differ in that they operate by changing electrical resistance in direct proportion to temperature changes [2]. This relationship is inherently non-linear over wide ranges, though it is highly predictable. The resistance $R(t)$ at temperature t is governed by the Callendar-Van Dusen equation [3]

$$R(t) = R_0(1 + At + Bt^2) \tag{1}$$

for temperatures $t > 0$°C, where A and B are standard coefficients. This quadratic physical relationship serves as the theoretical justification for using LSQ models in this work.

While PT100 sensors offer high stability, the resistance change is relatively small ($\approx 0.385\,\Omega/$°C). Consequently, accurate reading requires precise signal conditioning to convert resistance changes into measurable voltage levels [4]. This paper aims to optimize this conversion by analyzing different hardware front-ends and software regression models [5] to find an optimal solution for high-precision, low-cost temperature sensing.

III. Hardware Setup

Two main analog front-ends were tested to evaluate the trade-off between circuit complexity and measurement accuracy. Table I lists the components required for both analog front-ends, detailing the specific parts used for high-accuracy and cost-effective implementations.

TABLE I: Component List

Component	Qty.	Description
Arduino Uno	1	Microcontroller for processing and control.
ADS1115 ADC	1	16-bit external ADC for high-precision voltage measurement.
PT100 RTD Sensor	1	Platinum resistance thermometer.
100Ω Resistor	1	Precision resistor for Wheatstone bridge.
4kΩ Resistors	2	Resistors for Wheatstone bridge.
JHD 162A Parallel LCD	1	For displaying the final temperature.
22kΩ Potentiometer	1	To adjust the contrast of the LCD.
Breadboard	1	For creating solderless circuit connections.
Jumper Wires	Set	For connecting all the components.
Pen Thermometer <= 0.1°C resolution	1	Used for training purpose and as a reference.
Kettle/Heater with water	1	To heat the water, so that training data corresponding to a wide range of temperatures can be obtained

- *High-Precision Wheatstone Bridge with ADC:* This uses a Wheatstone bridge configuration coupled with an external 16-bit ADS1115 ADC [6]. This setup allows for differential measurement, significantly reducing noise and common-mode errors. The complete circuit schematic for the high-precision Wheatstone bridge setup is illustrated in Figure 1. Table II lists the connection details for integrating the external 16-bit ADS1115 ADC.

TABLE II: ADS1115 Connections

ADS1115 Pin	Arduino Pin	Purpose
VDD	5V	Power Supply
GND	GND	Common Ground
SCL	A5 (SCL)	I2C Clock Line
SDA	A4 (SDA)	I2C Data Line
A0	Input from Voltage Divider Node (between PT100 and R_REF)	Reading Voltage
A1	Junction between the two 4kΩ resistors	Reference for differential reading

- *Simple Voltage Divider:* This uses a simple voltage divider utilizing the Arduino's internal 10-bit ADC.

The Arduino-LCD connections are available in Table III.

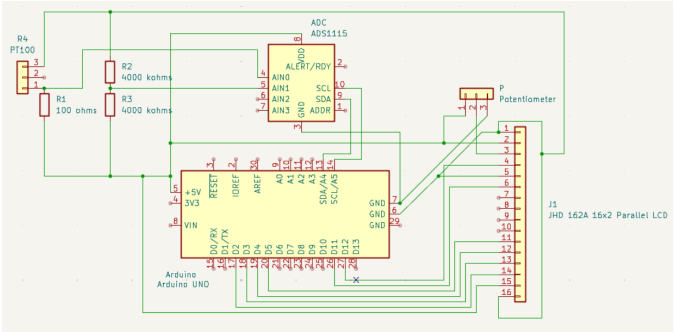

Fig. 1: Circuit Schematic of the High-Precision Wheatstone Bridge (Setup 1)

TABLE III: LCD Connections

LCD Pin	Arduino Pin
VSS	GND
VDD	5V
V0	Potentiometer Middle Pin
RS	Digital 12
RW	GND
E	Digital 11
D4	Digital 5
D5	Digital 4
D6	Digital 3
D7	Digital 2
A (Anode)	5V
K (Cathode)	GND

IV. Automated Training Using Optical Character Recognition (OCR)

The dataset is collected by simulating a real-world thermal change (cooling cycle) to capture paired temperature readings. Since the training data is available only on an analog thermometer, instead of manually noting the temperature, we automate the data collection process using OCR based techniques.

A. Data Collection Methodology

The experimental setup is available in Fig. 2. The practical requirements are listed below

1) *Hardware:*
   - A webcam connected to your computer.
   - **(Not necessary)** An NVIDIA GPU with CUDA installed. The script is pre-set to use `gpu=True` for much faster processing.
2) *Software:*
   - Python 3.x
   - The required Python libraries: `opencv-python`, `easyocr`, and `numpy`.

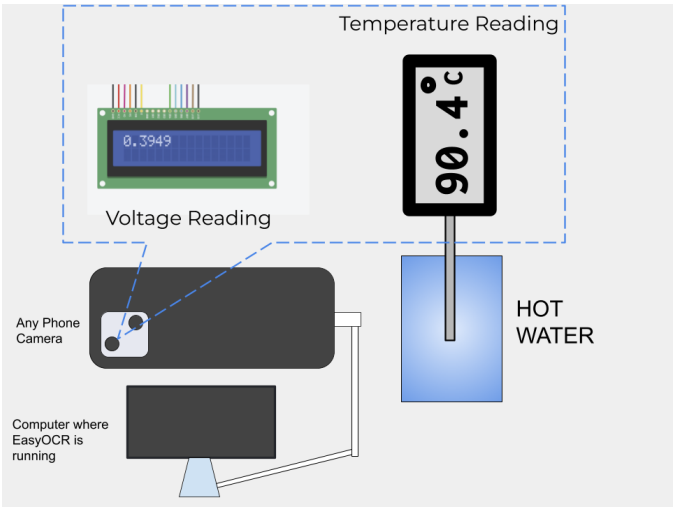

Fig. 2: Auto-Trainer representation

1) **Setup:** Place both the PT100 probe and the reference thermometer inside an electric kettle, submerged in water.
2) **Heating Phase:** Turn on the kettle and heat the water to its boiling point.
3) **Recording Phase:**
   - Switch off the kettle at boiling point.
   - Start recording a **video** using the smartphone.
   - Ensure **both the PT100 LCD display** and **reference thermometer** are visible in the frame.
4) **Cooling Phase:** Continue recording the cooling process as the water temperature drops naturally.
5) **Data Extraction:** The recorded video acts as the raw dataset.
   - Extract frames and apply **OCR (Optical Character Recognition)** to detect readings from both displays.

V. Introduction

This script uses OpenCV and EasyOCR to capture video from a webcam, identify and read text from two specific regions (voltage and temperature in this case), and log this data to a file.

It is designed to be calibrated for a specific, fixed-camera setup, such as this case where we are reading data from an LCD screen and a separate standard thermometer.

VI. Getting Started

Follow these instructions to get the script up and running on your local machine.

A. Prerequisites

Before you run the script, you'll need a few things set up.

A Note on the Webcam (Optional): This script will work with any webcam that OpenCV can detect, including built-in laptop cameras or standard USB webcams. The most important factor is a stable, well-positioned camera.

For reference, the development and testing for this project were done using a virtual camera setup:

- DroidCam: An application to use a smartphone as a high-quality webcam.
- OBS Studio: This software was used to capture the DroidCam feed and output it as an "OBS Virtual Camera," which was then selected by the Python script.

Again, this specific setup is not required. It is just one example of how to provide a camera feed to the script. Any simple webcam will work perfectly fine.

B. Installation

- **Create a Directory for Images:** The script is hardcoded to save debug images to an `img/` folder. You must create this folder in the same directory as the script for the program to run. Use the following command:

```
mkdir img
```

- **Install Python Libraries:** Install the necessary packages using `pip`:

```
pip install opencv-python easyocr numpy
```

Note: `easyocr` will also install `PyTorch`, which is a large download and a required dependency.

C. How to Run

1) Make sure your webcam is plugged in and positioned correctly to see both your target readings.
2) Open a terminal and navigate to the folder containing `a.py`.
3) Run the script:

```
python3 a.py
```

4) Two windows will appear: `'Voltage'` and `'Temperature'`. These show the exact (and processed) regions being sent to the OCR engine.
5) To stop the script, make sure one of these windows is active (click on it) and **press the 'q' key**.

D. Important: Calibration is Required!

This script **will not work correctly** without calibration. You must adjust the code to fit your specific camera, lighting, and display setup.

Here are the key sections you **must** modify in `a.py`:

1) GPU Usage: If you **do not** have a compatible NVIDIA GPU, you must change this line:

```
# From:
reader = easyocr.Reader(['en'], gpu=True)

# To:
reader = easyocr.Reader(['en'], gpu=False)
```

The script will run much slower on the CPU but will still function.

2) Frame Cropping: This is the most critical part. You need to tell the script exactly where to look for your data.

```
# Change the crop values according to requirement
voltage_frame = frame[0:int(h/2), :]
temp_frame = frame[int(h/2):, 0:int(w/3)]
```

Python slices are in [y1:y2, x1:x2] format (rows, then columns).

`voltage_frame` is currently set to the top half of the entire camera feed.

`temp_frame` is set to the bottom-left third of the feed.

You must change these pixel/percentage values to precisely isolate your voltage and temperature readings. The `Voltage` and `Temperature` windows will show you what the script sees in these cropped zones, so you can adjust the values and re-run until they are correct.

3) Image Processing:

- **Temperature**

```
# These values need to be calibrated according to
    requirement
gray_t = cv2.cvtColor(temp_frame, cv2.COLOR_BGR2GRAY)
t_lower = 50
t_upper = 100
_, thres = cv2.threshold(gray_t, 130, 255, cv2.
    THRESH_BINARY_INV)
kernel = cv2.getStructuringElement(cv2.MORPH_RECT, (11, 11))
closed_img = cv2.morphologyEx(thres, cv2.MORPH_CLOSE, kernel
    )
pretemp = cv2.bitwise_not(closed_img)
temp = cv2.GaussianBlur(pretemp, (17, 17), 0)
```

a) Key Values to Change::

- `cv2.threshold(gray_t,130,255, cv2.THRESH_BINARY_INV)`: The 130 is the threshold value. This is highly sensitive to lighting. Adjust it up or down until the text is clearly separated from the background in the `'Temperature'` window.
- `cv2.getStructuringElement(cv2.MORPH_RECT, (11,11))`: This is the kernel size for the `'closing'` operation, which helps connect broken parts of a number. You may need to make it larger or smaller.
- `cv2.GaussianBlur(pretemp, (17,17), 0)`: This is the kernel size for the blur.

Look at the `Temperature` window while calibrating. Your goal is to make the numbers you want to read clear and solid, while removing as much background noise as possible.

It is also preferred to hide the decimal point, either physically or by further image processing.

- **Voltage** In this case, the cropped voltage frame is converted to grayscale before it is used for OCR. Make sure the voltage reading is visible in the voltage frame. It is also preferred to hide the decimal point, either physically or by further image processing.

E. Output

The script generates two types of output:

- `out.txt`: A text file that logs the detected readings. Every 15 frames, if text is found in both regions, a

new line is added, formatted as: `[voltage_reading] [temperature_reading]`.

- `img/` Directory: This folder saves `voltageXX.png` and `tempXX.png` images. These are the exact frames that were sent to EasyOCR. If you are getting bad readings, check these images to see why. They are perfect for debugging your calibration and fixing the readings.

`out.txt` is not perfect due to some instances of incorrect character recognition. The data was manually updated and is saved in `trainingdata.txt`.

The final training data is in the following text file

https://github.com/gadepall/pt100/blob/main/AutoTrainer/trainingdata.txt

VII. Models Used

This report details the development and evaluation of several regression models for predicting sensor output. Specifically, we focus on two scenarios: predicting temperature from voltage using a Wheatstone bridge with ADC, and predicting resistance from voltage using a voltage divider without ADC. We investigate Random Forest Regressor [7], Inverse Quadratic Least Squares (LSQ), Direct LSQ, and Stochastic Gradient Descent (SGD) approaches. The models were implemented using Python and the scikit-learn. The source code corresponding to the training and evaluation pipelines are hosted at:

https://github.com/gadepall/pt100/blob/main/codes/models/PT100_models.ipynb

VIII. Model Evaluation and Comparison

This section presents the evaluation of the trained models and a comparison of their performance based on Mean Squared Error (MSE) and R-squared (R2) metrics.

A. Model Explanations

1) Inverse LSQ (Quadratic) for Voltage Divider without ADC: This model approximates the relationship between resistance (Y) and voltage (X) from the voltage divider circuit as $X = aY^2 + bY + c$. The fitted equation for the Voltage Divider is:

$$X = -0.000014990Y^2 + 0.005589496Y + 2.526066163$$

The coefficients are determined through linear least squares regression [5]. The model predicts resistance (Y) from measured voltage (X) using the positive root of the quadratic equation:

$$Y = \frac{-b \pm \sqrt{b^2 - 4a(c - X)}}{2a}$$

This method is suitable for circuits where the voltage-resistance relationship can be approximated quadratically.

2) Inverse LSQ (Quadratic) for Wheatstone with ADC: This model approaches the Wheatstone bridge problem by fitting a quadratic equation of the form $X = aY^2 + bY + c$, where Y is temperature and X is voltage. The fitted equation for the Wheatstone bridge is:

$$X = -0.000006744Y^2 + 0.004344843Y + 0.056019368$$

To predict Y from a given X, we use the quadratic formula:

$$Y = \frac{-b \pm \sqrt{b^2 - 4a(c - X)}}{2a}$$

This method aligns with the physical properties described by the Callendar-Van Dusen equation [3].

3) LSQ (Quadratic) for Wheatstone with ADC: This model applies a standard quadratic fit directly mapping voltage to temperature, expressed as $Y = aX^2 + bX + c$. Unlike the inverse models, this approach predicts temperature (Y) directly from the voltage (X) without requiring the use of the quadratic formula for inversion during the prediction phase.

- **Fitted Equation:** $Y = aX^2 + bX + c$
- **Coefficients:**

$$a = 190.226107678$$
$$b = 173.380527679$$
$$c = -7.072446221$$

4) Stochastic Gradient Descent (SGD) Regressor (Wheatstone with ADC): The SGD Regressor is a linear model trained with Stochastic Gradient Descent [7]. Due to its sensitivity to feature scaling, both the input features (X) and the target variable (Y) were scaled using StandardScaler before training. The predictions were then inverse-transformed back to the original scale for evaluation.

5) Inverse SGD Regressor (Wheatstone with ADC): This model utilizes Stochastic Gradient Descent to learn the inverse relationship, where voltage (X) is the target and temperature (Y) is the feature ($X = f(Y)$). Similar to the Inverse LSQ approach, the model learns to predict voltage from temperature. During inference, the temperature is derived by numerically inverting the learned function.

6) Random Forest Regressor (Wheatstone with ADC): The Random Forest Regressor is an ensemble learning method that constructs a multitude of decision trees during training [7]. For regression tasks, it averages the predictions of individual trees to produce a more stable and accurate prediction. This model was applied to predict temperature (Y) from voltage (X) directly, leveraging its ability to capture complex non-linear relationships without explicit feature engineering.

B. Performance Metrics Summary

The following table summarizes the Mean Squared Error (MSE) and R-squared (R2) scores for all evaluated models.

TABLE IV: Model Performance Metrics

Model	MSE	R2
1. Inverse LSQ (Volt. Div w/o ADC)	0.6612	0.9977
2. Inverse LSQ (Wheatstone w/ ADC)	0.009941	0.999953
3. LSQ (Wheatstone w/ ADC)	0.009198	0.999956
4. SGD Regressor (Wheatstone w/ ADC)	0.030477	0.999854
5. Inverse SGD (Wheatstone w/ ADC)	0.016221	0.999922
6. Random Forest (Wheatstone w/ ADC)	1.248897	0.994033

C. Validation Data and Model Predictions (Subset)

The below table presents a small subset of test data, showing the actual and predicted temperature values for the Voltage Divider setup.

TABLE V: Voltage Divider Validation Table with Predictions

Voltage	Actual Temperature	Prediction
2.7300	39.7	40.9900
2.7500	46.4	45.6500
2.8400	68.3	68.8900
2.8920	85.6	84.7100
2.9100	90.0	90.8000
2.8700	77.8	77.7400

The following table presents a small subset of the test data, showing the actual temperature values alongside the predictions from the Random Forest, Inverse LSQ, LSQ, SGD, and Inverse SGD Regressor models for the Wheatstone bridge setup with ADC.

TABLE VI: Wheatstone Validation Data and Predictions

Volt (V)	**Actual** (°C)	**Inv LSQ** (ADC)	**LSQ** (ADC)	**SGD** (ADC)	**Inv SGD** (ADC)	**RF** (ADC)
0.2634	51.9	51.9136	51.9070	51.7938	51.9906	53.3456
0.3127	65.9	65.7972	65.8329	65.7442	65.8829	67.6248
0.2083	37.2	37.1962	37.1906	37.2964	37.1134	37.0563
0.4037	93.1	93.6290	93.4194	93.9231	93.2383	89.0505
0.2092	37.5	37.4305	37.4238	37.5239	37.3513	37.0563
0.2978	61.6	61.5231	61.5485	61.4304	61.6219	61.8901
0.2028	35.8	35.7687	35.7707	35.9127	35.6628	36.7501
0.2666	52.7	52.7931	52.7892	52.6712	52.8748	53.3456
0.3185	67.5	67.4803	67.5183	67.4462	67.5568	68.7954
0.2850	57.9	57.9067	57.9202	57.7921	58.0053	58.6497

D. Graphical Comparison of Wheatstone ADC Models

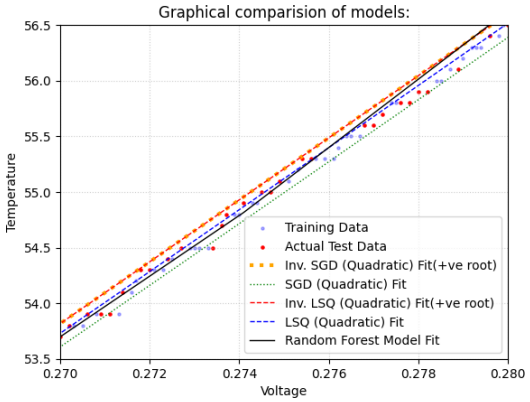

Fig. 3: Comparison of Random Forest, Inverse LSQ, LSQ, SGD and Inverse SGD Regressor predictions against actual test data for the Wheatstone bridge with ADC for temperatures between 53.5°C to 56.5°C

E. Model Comparison and Conclusion

Comparing the performance metrics, the Least Squares (Quadratic) model for the Wheatstone bridge achieved the highest R-squared score of 0.999956. It also achieved the lowest mean squared error of 0.009198. Qualitatively, we can say that the **Least Squares Quadratic** model is the best at predicting the temperature.

IX. Appendix - Arduino Deployment

Following are the steps to upload code to the Arduino

1) Connect Arduino to computer via USB
2) Upload the following code to the Arduino
   https://github.com/gadepall/pt100/blob/main/codes/arduino/code/code.ino

The Arduino code,

- Finds the voltage across PT100 through the ADC
- Predicts the temperature from the voltage with the model.
- Displays the temperature on the LCD display.

References

[1] T. Bartelt, *Industrial Control Electronics: Devices, Systems & Applications*. Delmar Cengage Learning, 2005.

[2] J. Fraden, *Handbook of Modern Sensors: Physics, Designs, and Applications*. Springer, 2016.

[3] H. L. Callendar, "On the practical measurement of temperature: Experiments made at the cavendish laboratory, cambridge," *Philosophical Transactions of the Royal Society of London. A*, vol. 178, pp. 161–230, 1887.

[4] T. Mien and N. Hien, "Precision temperature measurement using wheatstone bridge," in *2019 International Conference on System Science and Engineering (ICSSE)*. IEEE, 2019, pp. 145–149.

[5] A. M. Legendre, "The method of least squares," *Memoires de l'Academie Royale des Sciences*, 1805.

[6] *ADS111x Ultra-Small, Low-Power, I2C-Compatible, 860-SPS, 16-Bit ADCs*, Texas Instruments, 2018, sBAS444D.

[7] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg, J. Vanderplas, A. Passos, D. Cournapeau, M. Brucher, M. Perrot, and E. Duchesnay, "Scikit-learn: Machine learning in python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.